
DV2626/DV2599 Machine Learning

Lecture 4: Logical Models

Veselka Boeva

AI & Data Analytics Lab, Department of Computer Science, BTH

veselka.boeva@bth.se

(many slides from Niklas Lavesson)

Outline

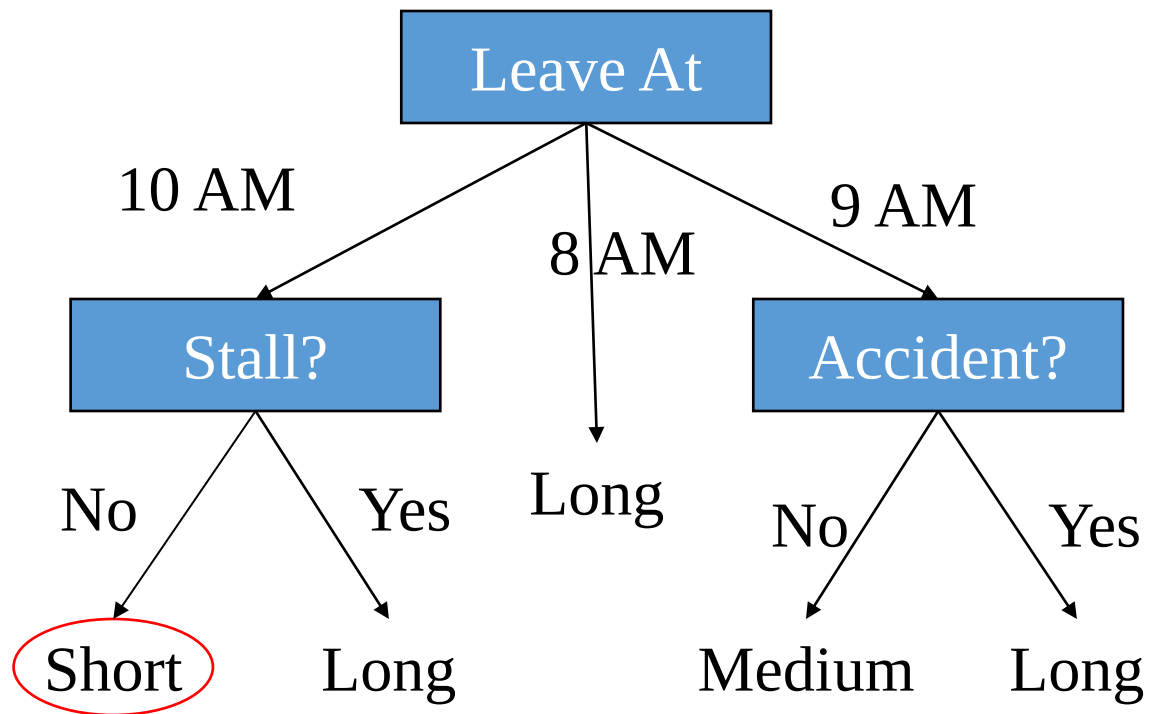
- Logical Models

- Tree models
- Rule models

Tree Models

- Tree models are
 - Expressive
 - Interpretable
- They are versatile and can be used for
 - Classification, regression, clustering, probability estimation, and many other machine learning tasks
- Due to the expressiveness of the tree model, tree learners are prone to overfitting

Decision tree example (1)



Predicting Commute Time

If we leave at 10 AM and there are no cars stalled on the road, what will our commute time be?

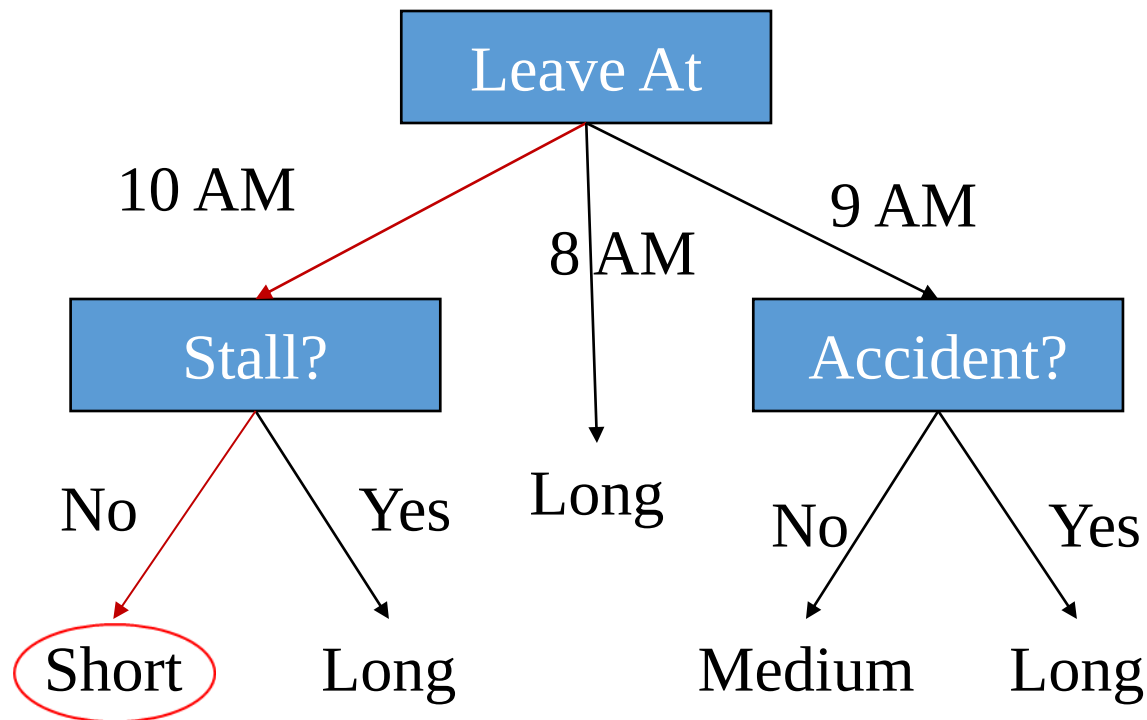
Decision tree example (2)

- In this decision tree, we made a series of Boolean decisions and followed the corresponding branch
 - Did we leave at 10 AM?
 - Did a car stall on the road?
 - Is there an accident on the road?
- By answering each of these yes/no questions, we then came to a conclusion on how long our commute might take

Basic Tree Learning

- The basic tree structure and learner is the same regardless of which task is addressed (see Definition 5.1 in the main literature)
- Each internal node (all nodes except the leaf nodes) is labelled with a feature
- Branching is conducted on feature-value pairs (literals), e.g.: Leave at = 8 AM, Accident = Yes, Leave at = 9AM
- The set of literals at a node is called a split
- Tree learning uses top-down search to identify the best hypothesis, given a task and observations

Decision tree example

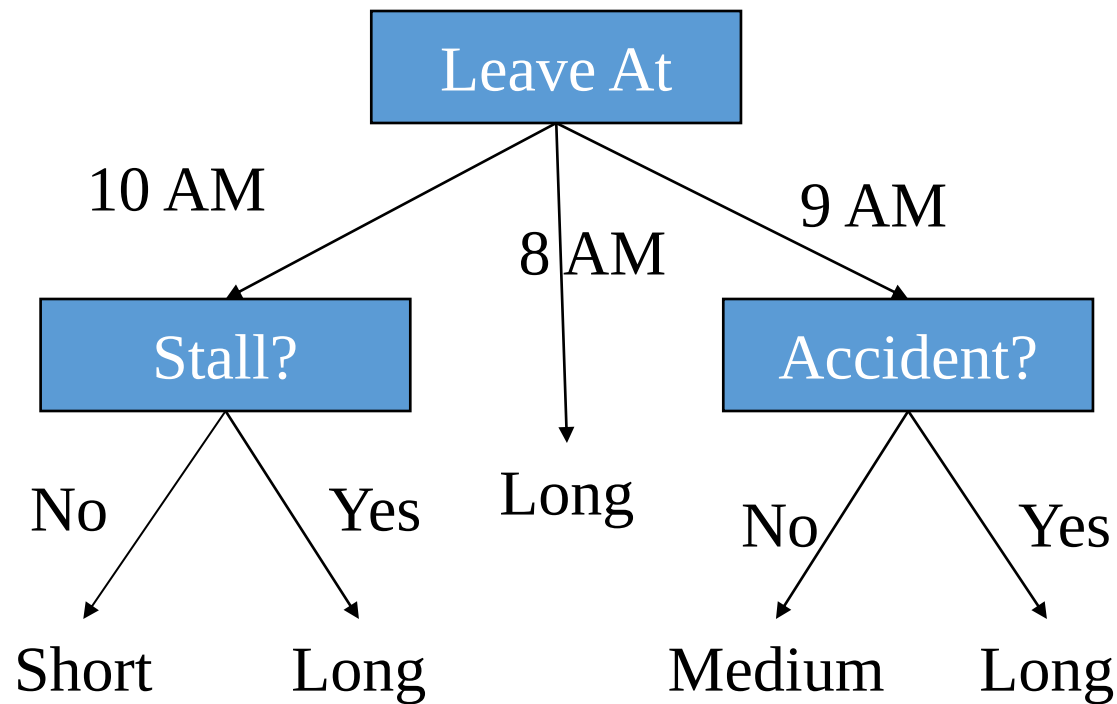


Predicting Commute Time

If we leave at 10 AM and there are no cars stalled on the road, what will our commute time be?

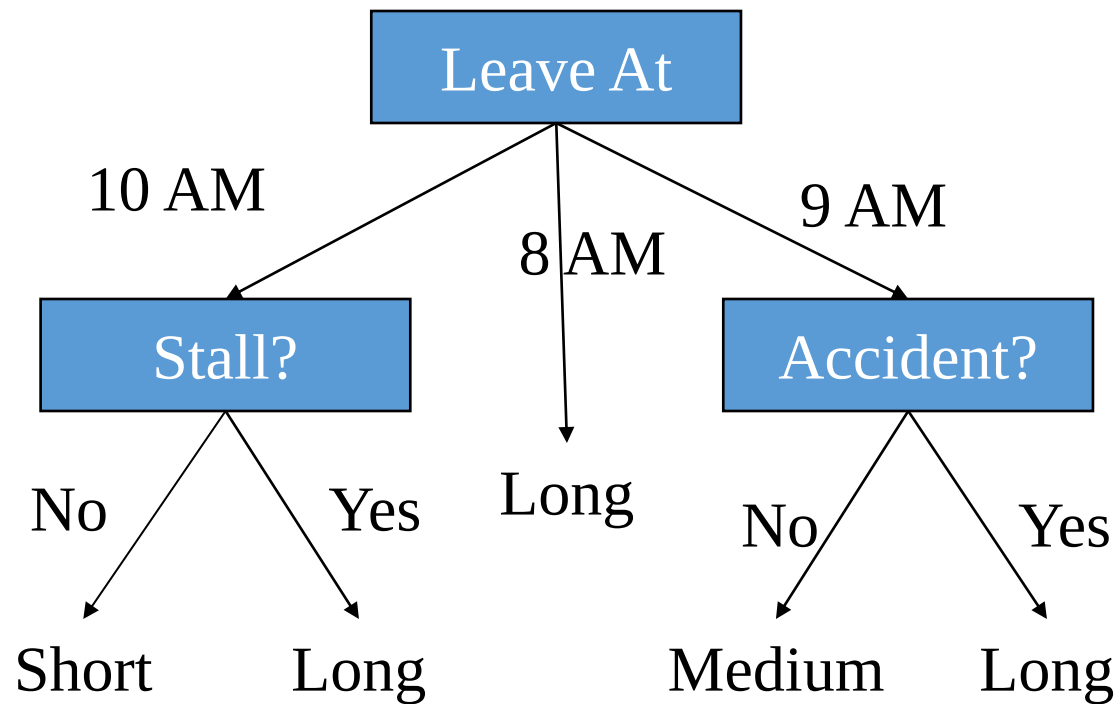
- A conjunctive hypothesis $\text{Short} = (\text{Leave At} = 10 \text{ AM}) \text{ and } (\text{Stall} = \text{No})$
- The set of instances covered by the conjunctive hypothesis is called the **instance space segment** associated with the leaf

Decision Trees as Rules



- We did not have to represent the tree graphically
- We could have represented it as a set of rules. However, this may be much harder to read...

Decision Trees as Rules



```
if hour == 8am
    commute time = long
else if hour == 9am
    if accident == yes
        commute time = long
    else
        commute time = medium
else if hour == 10am
    if stall == yes
        commute time = long
    else
        commute time = short
```

- Notice that all attributes do not have to be used in each path of the decision.
- As we will see, all attributes may not even appear in the tree.

Basic algorithm

- **func** *homogeneous* (*D*) returns true if the instances in *D* are alike enough to be labelled identically
- **func** *label* (*D*) returns the most suitable label for *D*
- **func** *bestsplit* (*D*, *F*) returns the best set of literals

```
func growtree (D, F) // a tree T with labelled leaves
  if homogeneous (D) then return label (D);
  S  $\leftarrow$  bestsplit (D, F); // e.g., BestSplit-Class algorithm
  split D into subsets Di based on S;
  for each i do
    if Di  $\neq \emptyset$  then Ti  $\leftarrow$  growtree (Di, F);
    else Ti  $\leftarrow$  label (D);
  end
return T
```

Task-specific function definitions

- Each function must be defined to suit the task at hand
- For **homogeneous** (D):
 - Classification: true if instances are mostly from the same class
 - Clustering: true if instances are close together
- For **label** (D):
 - Classification: returns the most probable class label
 - Regression: returns an average prediction of the homogeneous instances
 - Clustering: returns the cluster exemplar such as the mean

Decision trees (1)

- Decision trees are tree induction algorithms and models for classification tasks
- The term *inductive bias* refers to a set of (explicit or implicit) assumptions made by a learning algorithm in order to generalize a finite set of observation (training data) into a general model of the domain.
- Without a bias of that kind, induction would not be possible, since the observations can normally be generalized in many ways.

Decision trees (2)

- The inductive bias, the heuristic for generating a good hypothesis, is often designed to find a simple, accurate tree
- There is often a trade-off between accuracy and simplicity
- The higher accuracy we need, the more we may need to grow the tree to make sure the leaf nodes are pure (i.e., perfectly homogeneous)
- The assumption is that a simple tree that scores fairly well on training data will generalise well to unseen test data of the same kind

Computing decision tree splits (1)

func **bestsplit-class** (D, F)

$I_{\min} \leftarrow 1;$

for each $f \in F$ **do**

split D into subsets $D_1 \dots D_l$ based on the values v_i of f

if $\text{imp}(\{D_1, \dots, D_l\}) < I_{\min}$ **then**

$I_{\min} \leftarrow \text{imp}(\{D_1, \dots, D_l\});$

$f_{\text{best}} \leftarrow f;$

end

end

return f_{best}

- So how do we compute the impurity (imp) of node or split?

Computing decision tree splits (2)

- The impurity of a split is the weighted average of the impurities of its nodes:

$$\text{imp}(\{D_1, \dots, D_l\}) = \sum ((D_i/D) \text{imp}(D_i))$$

- The impurity of a node can be computed using different measures:
 - Minority class
 - Gini index
 - Information entropy
- **Example:**
 - *Minority class* - the fraction of mislabelled instances if all instances in a node are labelled as the majority class: $1/2 - |p - 1/2|$
 - *Gini index* - the expected error if we label examples in the leaf randomly: positive with probability p and negative with probability $1 - p$: $2p(1 - p)$
 - *Entropy* $-p \log_2 p - (1 - p) \log_2 (1 - p)$

p is the empirical probability of positive class

How to create a decision tree

- We first make a list of features (attributes) that we can measure
 - These attributes (for now) must be discrete
- We then choose a *target attribute* that we want to predict
- Then create an *experience table* that lists what we have seen in the past

Sample experience table

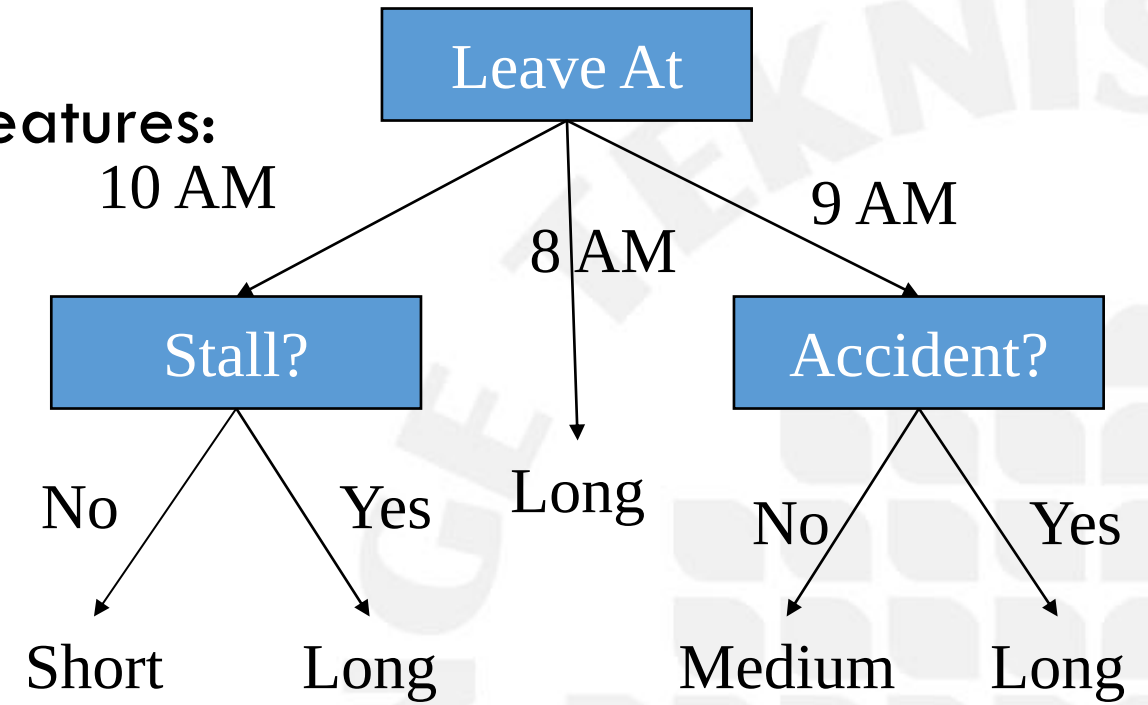
Example	Attributes				Target
	Hour	Weather	Accident	Stall	Commute
D1	8 AM	Sunny	No	No	Long
D2	8 AM	Cloudy	No	Yes	Long
D3	10 AM	Sunny	No	No	Short
D4	9 AM	Rainy	Yes	No	Long
D5	9 AM	Sunny	Yes	Yes	Long
D6	10 AM	Sunny	No	No	Short
D7	10 AM	Cloudy	No	No	Short
D8	9 AM	Rainy	No	No	Medium
D9	9 AM	Sunny	Yes	No	Long
D10	10 AM	Cloudy	Yes	Yes	Long
D11	10 AM	Rainy	No	No	Short
D12	8 AM	Cloudy	Yes	No	Long
D13	9 AM	Sunny	No	No	Medium

Choosing attributes (features) (1)

- The experience decision table showed 4 attributes: hour, weather, accident and stall

- But the decision tree only showed 3 features: hour, accident and stall

- Why is that?



Choosing attributes (features) (2)

- Methods for selecting attributes show that weather is not a discriminating attribute
- We use the principle of *Occam's Razor*: Given a number of competing hypotheses, the simplest one is preferable

Choosing attributes (features) (3)

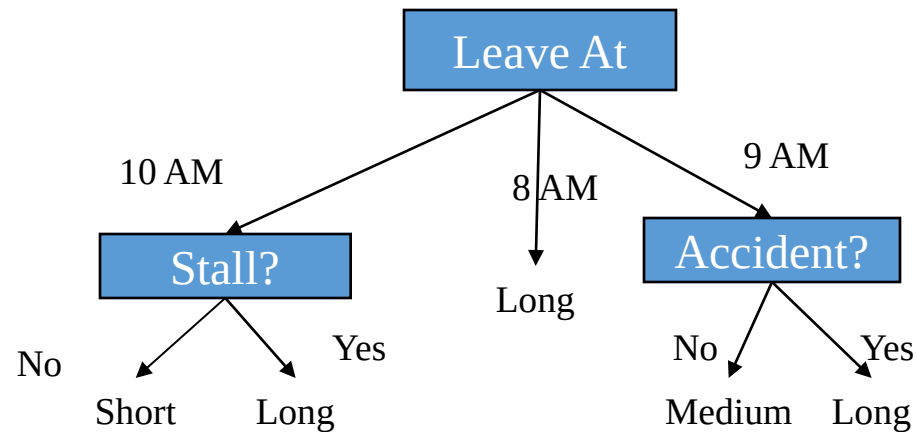
- The basic structure of creating a decision tree is the same for most decision tree algorithms
- The difference lies in how we select the features (attributes) for the tree
- We will focus on the ID3 (Iterative Dichotomiser 3, precursor to C4.5) algorithm developed by Ross Quinlan in 1975

Decision Tree Algorithms

- The basic idea behind any decision tree algorithm is as follows:
 - Choose the *best* attribute(s) to split the remaining instances and make that attribute a decision node
 - Repeat this process recursively for each child
 - Stop when:
 - All the instances have the same target attribute value
 - There are no more attributes
 - There are no more instances

Identifying the best attributes

- How did we know to split on *leave at* and then on *stall* and *accident* and not weather?



- ID3 selects attributes based on their *entropy*
- Entropy is the measure of disinformation...

Entropy (1)

- Entropy is minimized when all values of the target attribute are the same
 - If we know that commute time will always be *short*, then $\text{entropy} = 0$
- Entropy is maximized when there is an equal chance of all values for the target attribute (i.e. the result is random)
 - If commute time = short in 3 instances, medium in 3 instances and long in 3 instances, entropy is maximized

Entropy (2)

- Calculation of entropy
 - $\text{Entropy}(D) = -\sum_{(i=1 \text{ to } m)} |D_i|/|D| * \log_2(|D_i|/|D|)$
 - D = set of examples
 - D_i = subset of D with value v_i under the target attribute, i.e. $|D_i|/|D|$ is the proportion of cases (out of m) that belong to class v_i
 - m = size of the range of the target attribute
- Entropy ranges between 0 (most pure) and $\log_2(m)$ (equal representation of classes)

ID3 (1)

- ID3 splits on attributes with the lowest entropy
- We calculate the entropy for all values of an attribute as the weighted sum of subset entropies as follows:
 - $\sum_{(i = 1 \text{ to } k)} |D_i| / |D| \text{ Entropy}(D_i),$
where k is the range of the attribute we are testing
- We can also measure information gain (which is inversely proportional to entropy) as follows:
 - $\text{Entropy}(D) - \sum_{(i = 1 \text{ to } k)} |D_i| / |D| \text{ Entropy}(D_i)$

ID3 (2)

- Given our commute time sample set, we can calculate the entropy of each attribute at the root node

Attribute	Expected Entropy	Information Gain
Hour	0.6511	0.768449
Weather	1.28884	0.130719
Accident	0.92307	0.496479
Stall	1.17071	0.248842

Pruning Trees

- There is another technique for reducing the number of attributes used in a tree – *pruning*
- Two types of pruning:
 - Pre-pruning (forward pruning)
 - Post-pruning (backward pruning)

Pre-pruning

- In pre-pruning, we decide during the building process when to stop adding attributes (possibly based on their information gain)
- However, this may be problematic – Why?
 - Sometimes attributes individually do not contribute much to a decision, but combined, they may have a significant impact

Postpruning

- Postpruning waits until the full decision tree has built and then prunes the attributes
- Two techniques:
 - Subtree Replacement
 - Subtree Raising

Continuous Attribute?

- Each non-leaf node is a test, its edges partitioning the attribute into subsets (easy for discrete attribute).
- For continuous attribute
 - Partition the continuous value of attribute A into a discrete set of intervals
 - Create a new boolean attribute A_c , looking for a threshold c ,

$$A_c = \begin{cases} true & \text{if } A < c \\ false & \text{otherwise} \end{cases}$$

How to choose c ?

Strengths of Decision Trees

- can generate understandable rules
- perform classification without much computation
- can handle continuous and categorical variables
- provide a clear indication of which features are most important for prediction or classification

Weakness of Decision Trees

- Perform poorly with many classes and small data.
- Computationally expensive to train.
 - At each node, each candidate splitting attribute must be sorted before its best split can be found.
 - In some algorithms, combinations of features are used and a search must be made for optimal combining weights.
 - Pruning algorithms can also be expensive since many candidate sub-trees must be formed and compared.
- Do not treat well non-rectangular regions.

Rule Models

- Rule models offer more flexibility than tree models
- Two approaches to supervised rule learning
 - **rule list** - find the *body* of the rule that covers a sufficiently homogeneous set of examples, and find a label to put in the *head* of the rule
 - **rule set** - first select a class you want to learn, and then find rule bodies that cover the examples of that class

Learning ordered rule lists

- The idea of ordered rule learning is to keep growing a conjunctive rule body by adding the literal that most improves its homogeneity
- The key difference to decision tree learning is that we are only interested in the impurity of the instances covered by the rule (that is, one out of two nodes) not the average impurity
- While the introduced impurity measures may yield slightly different decision trees, they all yield the same rules
- After generating a rule that covers some (but not all) instances with minimal impurity, we remove the covered instances and proceed by generating a new rule, and so on
- The ordered rules can be viewed as an IF, ELSE IF, ELSE construction where the last part, ELSE, is referred to as the default rule